



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO

DISCIPLINA: LABORATÓRIO DE ALGORITMOS E ESTRUTURAS DE DADOS II

DOCENTE: KENNEDY REURISON LOPES

DISCENTES: ERYK DE FREITAS ALVES

JOSÉ RAFAEL DIAS PESSOA

RAQUEL CAVALCANTE DA SILVA

SISTEMA DE VERIFICAÇÃO DE CADASTRO COM TABELA HASH E FILTRO DE
BLOOM

PAU DOS FERROS - RN

29/06/2026

SUMÁRIO

- 1. INTRODUÇÃO.....2
- 2. IMPLEMENTAÇÃO..... 2
 - 2.1 Tabela Hash..... 2
 - 2.2 Filtro de Bloom..... 3
 - 2.3 Integração (main.c)..... 3
- 3. FALSOS POSITIVOS..... 4
- 4. EXPERIMENTOS..... 4
 - 4.1 Metodologia..... 4
 - 4.2 Resultados..... 4
 - 4.3 Análise dos resultados..... 5
- 5. CONCLUSÃO..... 6
- 6. REFERÊNCIAS..... 7

1. INTRODUÇÃO

O crescimento do volume de dados em sistemas digitais exige estruturas capazes de responder consultas de existência com rapidez e baixo consumo de memória. Este trabalho implementa um Sistema de Verificação de Cadastro de Usuários que combina duas estruturas complementares: uma Tabela Hash, responsável pelo armazenamento e recuperação exata dos registros, e um Filtro de Bloom, utilizado como pre-filtro probabilístico para evitar consultas desnecessárias à estrutura principal.

O objetivo é avaliar experimentalmente o impacto dessa combinação em termos de desempenho (tempo de consulta), consumo de memória e taxa de falsos positivos para conjuntos de 1.000, 10.000 e 100.000 registros, discutindo as vantagens e limitações de cada estrutura.

2. IMPLEMENTAÇÃO

2.1 Tabela Hash

A Tabela Hash foi implementada com a função hash DJB2, definida pela recorrência $\text{hash} = \text{hash} \times 33 + c$, onde c é o valor ASCII de cada caractere da string. O tratamento de colisões adota encadeamento externo: cada posição armazena o início de uma lista encadeada de nós.

O tamanho da tabela foi fixado em 150.001 posições, valor primo escolhido para minimizar colisões por agrupamento e garantir fator de carga (α) próximo de 0,67 para até 100.000 elementos. Esse intervalo é recomendado para encadeamento externo, pois mantém o comprimento médio das listas abaixo de 1, preservando o desempenho de busca em $O(1)$ amortizado. Nos testes com 100.000 registros, o fator de carga obtido foi de 0,666662, confirmando o dimensionamento.

2.2 Filtro de Bloom

O Filtro de Bloom utiliza um vetor de bits e aplica Double Hashing com as funções DJB2 e SDBM para calcular k posições distintas por elemento. O tamanho do vetor (m) e a quantidade de funções hash (k) são calculados automaticamente pelas fórmulas:

$$m = -(n \times \ln(p)) / (\ln 2)^2 \quad k = (m / n) \times \ln 2$$

Para $n = 100.000$ elementos esperados e $p = 1\%$ de falsos positivos, obtém-se $m = 958.506$ bits (aproximadamente 117 KB) e $k = 7$ funções hash. Esses valores foram validados experimentalmente, com taxas de falsos positivos entre 1,01% e 2,54% nos três cenários testados.

Uma correção importante implementada foi a guarda contra $h2 = 0$ no Double Hashing. Quando a função BDSM retorna zero, todas as k posições colapsam para a mesma, tornando o filtro inutilizável. O código garante $h2 \geq 1$ antes de calcular qualquer posição.

2.3 Integração (main.c)

O módulo principal implementar os quatro requisitos funcionais exigidos:

- RF01 — Inserção manual com validação de formato (8 letras minúsculas + 3 dígitos);
- RF02 — Consulta com fluxo obrigatório: Bloom primeiro, Hash somente se o Bloom indicar possível existência;
- RF03 — Estatísticas completas: total de consultas, evitadas pelo Bloom, falsos positivos, taxa percentual e tempo médio;
- RF04 — Inserção em lote a partir de arquivo texto, com validação de formato e contagem de inválidos ou duplicados.

3. FALSOS POSITIVOS

Um falso positivo ocorre quando o Filtro de Bloom indica que um elemento provavelmente existe no conjunto, mas a consulta subsequente na Tabela Hash confirma que ele não está cadastrado. Em outras palavras: o filtro mentiu positivamente.

Isso acontece porque o Bloom é uma estrutura probabilística. Ao inserir um elemento, ele liga k bits no vetor. Ao consultar outro elemento que nunca foi inserido, é possível que, por coincidência, todos os seus k bits já estejam ligados por elementos anteriores. Nesse caso o filtro retorna realmente incorretamente.

O impacto prático do falso positivo é que o sistema realiza duas operações (Bloom + Hash) para concluir uma ausência que deveria ter sido descartada só pelo Bloom. Isso

desperdiçou tempo e acesso à estrutura principal. Por isso, manter a taxa de falsos positivos baixa e fundamental, é exatamente o que o dimensionamento correto de m e k garante.

A taxa teórica de falsos positivos é dada por:

$$p = (1 - e^{(-kn/m)})^k$$

Com os parâmetros adotados ($m = 958.506$, $k = 7$, $n = 100.000$), a taxa teórica é de aproximadamente 1%. Os experimentos mediram taxas de 1,01% a 2,54%, dentro do intervalo esperado considerando a aleatoriedade dos dados gerados.

4. EXPERIMENTOS

4.1 Metodologia

Os experimentos foram executados com conjuntos de 1.000, 10.000 e 100.000 registros gerados aleatoriamente no formato [8 letras][3 dígitos], conforme especificado. Para cada cenário mediu-se: (a) o tempo de busca de todos os registros diretamente na Hash, sem usar o Bloom; (b) o tempo com o Bloom como pré-filtro; e (c) a taxa de falsos positivos, calculada sobre um conjunto de ausentes garantidos. Cada cenário foi executado de forma isolada, com tabela e Bloom recriados do zero para evitar contaminação entre testes.

4.2 Resultados

Registros	Sem Bloom	Com Bloom	Falsos positivos	Fator de carga
1.000	< 0,000001 s	< 0,000001 s	2,10%	0,006667
10.000	< 0,000001 s	0,008000 s	1,01%	0,066666
100.000	0,018000 s	0,048000 s	2,54%	0,666662

Tabela 1 — Resultados obtidos na execucao do programa.

4.3 Análise dos resultados

1. O Filtro de Bloom reduziu consultas na Tabela Hash?

Nos experimentos realizados, todos os elementos consultados haviam sido previamente inseridos. Para elementos existentes, o Bloom sempre retorna verdadeiro,

portanto 100% das consultas chegaram a Hash — nenhuma foi evitada. Isso é o comportamento correto e esperado do filtro: ele nunca produz falsos negativos, garantindo que elementos cadastrados sejam sempre encontrados. O ganho real do Bloom se manifesta quando há consultas a elementos ausentes, cenário em que o filtro descarta a maioria sem acionar a Hash. Em sistemas reais de verificação de cadastro, onde muitas tentativas são de usuários inexistentes, esse ganho é expressivo.

2. Por que o tempo com Bloom foi maior que sem Bloom?

Para elementos existentes, o fluxo com Bloom realiza duas operações: primeiro calcula $k = 7$ funções hash e verifica 7 bits no vetor, depois acessa a Tabela Hash. Já sem o Bloom, acessa a Hash diretamente. Portanto, para esse cenário específico, o Bloom adiciona trabalho em vez de economizar. No teste com 100.000 registros, o overhead de calcular 7 hashes para cada elemento resultou em tempo 2,6 vezes maior com Bloom (0,048 s) do que sem (0,018 s). Esse resultado é esperado e correto — ele não indica falha no sistema, mas sim que o benefício do Bloom está nas consultas negativas, não nas positivas.

3. Como o tamanho do vetor impactou os resultados?

O vetor de 958.506 bits foi dimensionado para 100.000 elementos. Nos cenários de 1.000 e 10.000 registros, o filtro operou subpopulação, o que explica taxas de falsos positivos próximas ou abaixo de 1%. No cenário de 100.000, a ocupação atingiu o nível projetado e a taxa ficou em 2,54%, ainda dentro do intervalo aceitável. Um vetor subdimensionado elevaria essa taxa rapidamente, pois mais bits estariam ligados por coincidência.

4. O número de funções hash alterou a precisão?

Com $k = 7$, o filtro distribuiu bem os bits pelo vetor, mantendo a taxa próxima da meta de 1%. Valores menores de k reduziram a precisão pois menos bits identificariam cada elemento; valores maiores aumentariam o tempo de operação e saturaram o vetor mais rapidamente, elevando a taxa de falsos positivos a longo prazo.

5. Em qual cenário o Bloom foi mais vantajoso?

Considerando o objetivo do sistema — verificar se um usuário está cadastrado — o Bloom é mais vantajoso quando o volume de consultas negativas é alto. O cenário de 100.000 registros é o mais representativo de um ambiente real, pois o fator de carga da Hash (0,67) e o da ocupação do Bloom estão no nível projetado. Nesse cenário, consultas a usuários

inexistentes seriam descartadas pelo Bloom em aproximadamente 97,5% dos casos (descontando os 2,54% de falsos positivos), evitando o acesso a Hash.

5. CONCLUSÃO

O sistema implementado demonstra que a combinação de Tabela Hash e Filtro de Bloom é eficaz para verificação de cadastro em larga escala. O Filtro de Bloom atua como barreira de primeira linha, descartando rapidamente elementos inexistentes sem acionar a estrutura principal. O dimensionamento adequado de m e k é fundamental: valores incorretos elevam a taxa de falsos positivos ou desperdiçam memória.

Os experimentos confirmaram que o filtro opera dentro da taxa de falsos positivos projetada (próxima de 1%) e que a Tabela Hash mantém fator de carga adequado (0,67) para o volume de dados esperado. A principal observação é que, para consultas a elementos existentes, o Bloom adiciona overhead em vez de economizar — o ganho real se dá nas consultas negativas, que são o cenário predominante em sistemas de verificação de usuários em produção.

Como trabalho futuro, sugere-se implementar o experimento com uma carga mista de consultas (existentes e ausentes) para quantificar o ganho do Bloom em condições mais próximas do uso real, além de comparar diferentes funções hash para avaliar seu impacto na distribuição e na taxa de falsos positivos.

6. REFERÊNCIAS

BERNSTEIN, Daniel J. Hash functions for hash table lookup. Disponível em: <http://www.cse.yorku.ca/~oz/hash.html> . Acesso em: 29 jun. 2026.

BLOOM, Burton H. Space/time trade-offs in hash coding with allowable errors. Communications of the ACM, New York, v. 13, n. 7, p. 422-426, jul. 1970.

CORMEN, Thomas H. et al. Introdução a Algoritmos. 3. ed. Rio de Janeiro: Elsevier, 2012. 928 p.

KNUTH, Donald Ervin. The Art of Computer Programming: sorting and searching. 2. ed. Reading: Addison-Wesley, 1998. v. 3. 780 p.

MITZENMACHER, Michael; UPFAL, Eli. Probability and Computing: randomized algorithms and probabilistic analysis. Cambridge: Cambridge University Press, 2005. 368 p.